

Assignment 4: A Small GPT

1. Goal

Your group builds a small GPT and trains it on the same BPE tokens. The neural n -gram of Assignment 3 could only see a fixed window of $n - 1$ tokens. This model sees every earlier position in the block, and self-attention lets it learn which of those positions matter.

You write the attention yourselves. At the end you put all four models in one table: the count n -gram, the neural n -gram, and the GPT, on Shakespeare and on the Wall Street Journal.

2. Rules

1. You may use PyTorch. The standard building blocks are all allowed: the embedding, linear, layer norm, dropout, module list and sequential containers, together with `F.softmax` and `F.cross_entropy`.
2. Do not use a ready-made attention or transformer layer, and do not use a pretrained model. This rule covers `nn.Transformer` and `nn.TransformerDecoder`, and it covers `nn.MultiheadAttention` and the fused attention function in `torch.nn.functional`. Write the attention yourselves, with matrix products and a mask.
3. Use your own BPE tokenizer and your own `perplexity` and `generate` functions. Do not rewrite them.
4. Use the validation set to choose hyperparameters. Use the test set once, at the end.
5. Make the run deterministic. Call `torch.manual_seed` and report the seed.
6. You may use an AI assistant. If you do, write one paragraph in the README that explains how it played a role in your implementation.

3. Data and device

Use the same Shakespeare splits and the same Wall Street Journal test file as in Assignment 3, prepared in the same way. Set the device once, as in Assignment 3:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
xb, yb = xb.to(device), yb.to(device)
```

Attention needs more computation than the neural n -gram, so the CPU configuration is small:

Setting	CPU (use this one)	GPU, if you have one
<code>block_size</code>	64	256
<code>n_embd</code>	128	384
<code>n_head</code>	4	6
<code>n_layer</code>	3	6
<code>batch_size</code>	32	64
<code>lr</code>	1e-3	3e-4
<code>dropout</code>	0.1	0.2
<code>max_steps</code>	2000	5000

The notebook that we run must use the CPU column, and one training run must finish in under 10 minutes. If you also ran the GPU column, report those numbers in a separate table and say which machine you used. Note that `n_embd` must divide by `n_head`.

4. Task 1: Blocks of tokens (10 points)

The model reads a block of `block_size` tokens and predicts the next token at every position in the block. One block of 64 tokens is therefore 64 training examples, not one.

1. Write `get_batch(split)`. Choose `batch_size` random start positions in the token list. Return `x` and `y`, both of shape `(batch_size, block_size)` and both on the device. Here `y` is `x` shifted by one position.
2. Print one small batch and check by hand that the target at position `t` is the token at position `t + 1` of the input.

5. Task 2: Self-attention (30 points)

1. Write one head. Three `nn.Linear` layers with `bias=False` produce the query, the key and the value from the input. The affinities are the query times the transposed key, scaled by the inverse square root of the head size.
2. Apply the causal mask. Build a lower triangular matrix with `torch.tril`, register it with `register_buffer`, and use `masked_fill` to write `-inf` above the diagonal. Then apply `F.softmax` along the last axis, and multiply by the value.
3. Write `MultiHeadAttention`: several heads in an `nn.ModuleList`, their outputs joined with `torch.cat` along the last axis, then one `nn.Linear` projection and `nn.Dropout`.
4. Write `FeedForward`. One linear layer widens the input to four times `n_embd`, then `nn.ReLU`, then a second linear layer maps it back, then dropout.
5. Write `Block`. It applies attention and then the feedforward network, each inside a residual connection, with a `nn.LayerNorm` before each one.

Print the shape after every step for one batch, and check it against what you expect. The mask is the part that fails silently, so slice it as `tril[:T, :T]`: during generation the context is shorter than `block_size`.

6. Task 3: The model and training (25 points)

1. Write the GPT module: a token embedding table, a position embedding table, the blocks in an `nn.Sequential`, a final `nn.LayerNorm`, and a linear layer to the vocabulary size. Add the token and position embeddings; do not concatenate them.
2. `forward` returns the logits and the loss, with the same reshape before `F.cross_entropy` that you used for the bigram model in the lecture.
3. Print the number of parameters. `Sum p.numel() over model.parameters()`.
4. Train with `torch.optim.AdamW`. Reuse the evaluation loop from Assignment 3, with the no-gradient decorator and the switch between `model.eval()` and `model.train()`.
5. Plot the training loss and the validation loss against the step number.

Two checks. Before training, the loss should be near $\ln V$. Print it. A much lower value means the mask is not working and the model is reading the answer. After training, the validation loss must be below the training loss of the neural n -gram at the same k , or something is wrong.

The interface. Add `next_token_log_probs(context)` and `log_prob(token, context)`, with the signatures from Assignment 2. Crop the context to the last `block_size` tokens inside these methods. Your `perplexity` and `generate` functions then run unchanged.

7. Task 4: Experiments (25 points)

Report the perplexity per token and the perplexity per character. Use the perplexity per character whenever you compare across k .

Experiment 1, vary k . Train the GPT with $k \in \{250, 1000, 4000\}$, everything else fixed. Report both perplexities on the validation set and the test set. Compare the shape of the curve against the one you found in Assignment 3, in two sentences.

Experiment 2, two hyperparameters. Choose two from this list: `n_layer`, `n_head`, `n_embd`, `block_size`, learning rate, dropout. Try three values of each, one at a time, while the other stays at the CPU default. That is six runs. Report the validation perplexity of each and plot one line per hyperparameter. Say which of the two matters more, and what each one costs in time or memory.

Experiment 3, the final comparison. Put every model of the course in one table:

Model	Shakespeare PP	PP per char	WSJ PP per char	Parameters	Train time
Count n -gram (A2)					
Neural n -gram (A3)					
GPT (A4)					

Use the same k for all three rows. Write one short paragraph on what the table shows: where the largest gain appears, and whether the gap on the Wall Street Journal is the same as the gap on Shakespeare.

8. Task 5: Generation (10 points)

1. Generate ten samples with `mode="sample"` and three with `mode="argmax"`, using the same three prompts as in Assignment 3.
2. Put one sample from each of the three models side by side, for the same prompt. Comment in three sentences: one per model, on what each one gets right and wrong.
3. Say whether the model stops at `<eos>` on its own, and how often.

9. Required interface

```
from bpe import BPETokenizer
from evaluation import perplexity, generate      # unchanged since Assignment 2
from gpt import GPT

model = GPT(vocab_size=V, n_embd=128, n_head=4, n_layer=3,
            block_size=64, dropout=0.1).to(device)
train(model, train_ids, max_steps=2000, lr=1e-3, batch_size=32)

model.next_token_log_probs(context_ids)         # numpy array, length V
pp, pp_char = perplexity(model, test_ids)
text = generate(model, tok, "shall i compare thee", mode="sample", max_tokens=100, seed=0)
```

10. What you hand in

The deadline for all submissions is 30 September 2026 at 17:00. This applies to Assignment 1 to Assignment 4.

Make one ZIP file with the name `group_NN_a4.zip`. Upload it to the folder of your group on StudIP. One member uploads for the whole group. The ZIP contains:

- `assignment4.ipynb`, with all output cells kept. It must run from top to bottom after *Restart and Run All*, on a CPU.
- Your `.py` files, including the ones you reuse from earlier, and `requirements.txt`.
- `README.md`, with the contribution table described below. Keep the ZIP under 10 MB. Do not hand in model checkpoints larger than that.

The contribution statement. The `README.md` must start with a table in this form:

Name	Student number	Tasks	Share
Student 1	123456	Task 2, Task 3	40 %
Student 2	234567	Task 4, plots	35 %
Student 3	345678	Task 1, Task 5, README	25 %

The shares must sum to 100. All members must agree to the table. Add one sentence about the work of each member in your own words. We grade the submission as one piece of work, but we use this table in the review session, and we can adjust a single grade if the table and the review do not agree. A submission without this table loses 10 points.

The `README.md` also gives the total run time, the machine and the device that you used, the random seed, and one paragraph on how AI played a role in your implementation, if you used it. If the full run takes more than 40 minutes, save the trained models and read them back, and add a flag `FORCE_RETRAIN = False` at the top of the notebook.

11. Grades

Item	Points
Task 1. Blocks and shifted targets, checked by hand	10
Task 2. Self-attention with a working causal mask, multi-head, block	30
Task 3. The model, the training loop, the two checks, the interface	25
Task 4. The three experiments, with tables and plots	25
Task 5. Generation and the comparison of the three models	10
Total	100

We remove points for these problems: the notebook does not run from top to bottom, -15 ; it does not run on a CPU, -10 ; a ready-made attention layer was used, -20 ; the contribution table is absent, -10 ; the old evaluation functions were rewritten instead of reused, -10 ; the test set is used to choose hyperparameters, -10 ; a plot has no axis label or no legend, -2 each.

12. After this assignment

You now have the whole path in your own code: characters to subwords, counts to a network, a fixed window to attention. The model in this assignment is the same architecture that large language models use. Only the size of the data, the size of the model and the length of the training run are different.

If you want to continue on your own, the useful next steps are a larger corpus, a longer block size, and weight tying between the token embedding and the output layer.